

Day 23 – MLflow, MLOps and LLMOps

1. Current MLflow baseline

As of **July 30, 2026**, the latest MLflow documentation targets **MLflow 3.14.0**. MLflow 3.x treats GenAI applications as first-class systems: experiments, traces, prompts, evaluation datasets, feedback, scorers, registered models and deployment evidence can be connected through a common lineage model. Recent releases also added capabilities such as workspaces, stronger RBAC, trace lifecycle management, review workflows and an official Helm deployment path. ([MLflow AI Platform](#))

Important modern terminology:

Prefer	Avoid for new systems
Model aliases such as candidate and champion	Registry stages such as Staging and Production
<code>mlflow.genai.evaluate()</code> and GenAI scorers	Treating all LLM evaluation as classic <code>mlflow.evaluate()</code>
Prompt Registry versions and aliases	Prompts stored only in source code
MLflow traces and spans	Logging an entire RAG request as one flat JSON artifact
Immutable model, prompt and index references	Runtime dependence on a mutable <code>latest</code> reference
SQL-backed MLflow server	Shared production deployment using local FileStore

Model Registry stages have been deprecated since MLflow 2.9. New implementations should use aliases, tags and deployment-environment separation instead. ([MLflow AI Platform](#))

2. Mapping the lifecycle onto MLflow

Use this production RAG application as the running example:

- FastAPI RAG service.
- Managed LLM or a model served through vLLM/KServe.
- Embeddings stored in a vector database.
- Documents and evaluation datasets stored in S3.
- MLflow on EKS with PostgreSQL and S3.
- Prefect for ingestion and evaluation orchestration.
- Existing CI for quality gates.
- Argo CD for GitOps deployment.
- OpenTelemetry, Prometheus and Grafana for operational observability.

MLflow implements the lifecycle as follows:

Existing lifecycle activity	MLflow implementation
Experiment with chunking, prompts and models	Experiments, runs, parameters, nested runs
Preserve outputs	Artifacts in S3

Identify input datasets	Dataset objects, digests, source metadata and manifests
Compare RAG quality	GenAI evaluation runs and scorers
Debug RAG requests	Traces with retriever, reranker, LLM and guardrail spans
Manage prompts	Prompt Registry versions, tags and aliases
Manage packaged models/applications	Logged Models and Model Registry
Collect human review	Feedback assessments attached to traces
Preserve release evidence	Run metrics, evaluation tables, artifacts and registry tags
Promote approved versions	Registry aliases plus immutable Git deployment references
Understand deployment lineage	Git SHA, image digest, model version, prompt version and index digest tags

The implementation goal is not merely “tracking experiments.” It is being able to answer:

Which source documents, index, embedding model, prompt, application image, model and evaluation evidence produced the answer currently served in production?

3. Responsibility boundaries

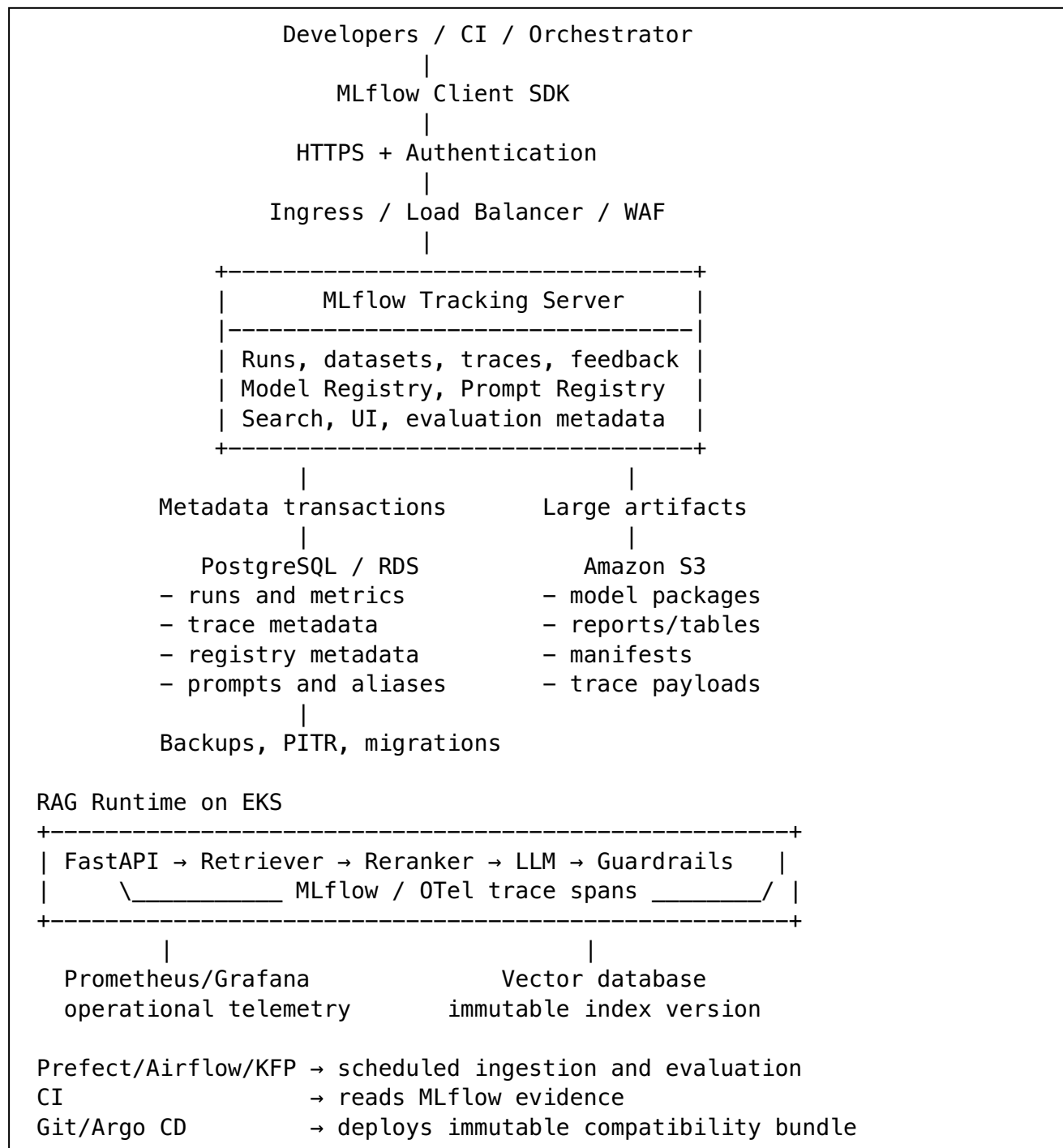
MLflow is an **AI lifecycle system**, not an all-purpose platform replacement.

System	Owns	Does not own
MLflow	Experiment evidence, artifacts, datasets, prompts, models, traces, feedback, evaluation and lineage	Scheduling, Kubernetes reconciliation, infrastructure alerts
Prefect/Airflow/ KubeFlow	Workflow dependencies, schedules, triggers, retries, backfills and recovery	Deciding whether a candidate meets a quality threshold
CI	Tests, scans, reproducible builds and release-gate execution	Continuously reconciling Kubernetes
Argo CD	Git-to-cluster reconciliation, drift control and rollback	Model evaluation or experiment tracking
vLLM/KServe/ provider API	Runtime inference and serving behavior	Experiment comparison and prompt governance
Prometheus/ Grafana	Real-time metrics, dashboards, SLOs and alerts	Detailed experiment lineage
OpenTelemetry	Cross-service telemetry propagation and collection	Prompt or model promotion policy
S3/PostgreSQL	Durable object and metadata storage	Lifecycle semantics by themselves

A useful senior-level statement is:

The orchestrator decides **when and in what order** work runs. MLflow records **what happened and what was produced**. CI decides **whether a candidate is releasable**. Argo CD decides **whether the cluster matches Git**.

4. Production architecture



The MLflow Tracking Server is an HTTP service with a UI and REST APIs. Metadata such as parameters, metrics, tags, registry records and trace identifiers belongs in the backend store. Large objects such as models and reports belong in the artifact store. SQL backends including PostgreSQL are supported; FileStore is now a legacy path rather than the recommended shared production backend. ([MLflow AI Platform](#))

5. Progressive hands-on implementation

5.1 Start MLflow locally

For a pinned learning environment:

```
python -m venv .venv
source .venv/bin/activate

pip install \
    "mlflow==3.14.0" \
    pandas \
    scikit-learn \
    openai

mlflow server \
    --host 127.0.0.1 \
    --port 5000
```

Then open the MLflow UI at port 5000.

The official documentation also provides a Docker Compose development environment using PostgreSQL and MinIO. That is preferable when you want local behavior closer to a production SQL-plus-object-store deployment. ([MLflow AI Platform](#))

5.2 Define the tracking contract

Do not allow every engineer to invent arbitrary names. Define standard parameters, tags, metrics and artifacts.

```
from __future__ import annotations

import hashlib
import json
import os
from dataclasses import asdict, dataclass

import mlflow
import pandas as pd

mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("customer-support-rag")

@dataclass(frozen=True)
class RAGConfig:
    llm_provider: str
    llm_model: str
    prompt_version: int
    embedding_model: str
    chunk_size: int
    chunk_overlap: int
    retriever_type: str
    top_k: int
    reranker_model: str
    temperature: float
    index_version: str
    source_dataset_version: str
```

```
def sha256_json(value: object) -> str:
    payload = json.dumps(
        value,
        sort_keys=True,
        separators=(",", ":"),
    ).encode("utf-8")
    return hashlib.sha256(payload).hexdigest()
```

Recommended naming convention:

Parameters:

- llm.provider
- llm.model
- llm.temperature
- prompt.version
- embedding.model
- chunk.size
- chunk.overlap
- retriever.type
- retriever.top_k
- reranker.model

Tags:

- git.sha
- image.digest
- environment
- service
- owner
- index.version
- source_dataset.version
- release.candidate
- risk.class

Metrics:

- eval.correctness
- eval.groundedness
- eval.retrieval_relevance
- eval.safety
- latency.p50_ms
- latency.p95_ms
- tokens.input_total
- tokens.output_total
- estimated_cost_usd
- error_rate
- feedback.positive_rate

Parameters should describe configuration. Metrics should represent measured numeric results. Tags should capture searchable identity and governance metadata. Artifacts should store structured reports that would be awkward or lossy as parameters.

5.3 Log parameters, metrics, tags, datasets and artifacts

```
config = RAGConfig(
    llm_provider="openai",
    llm_model="gpt-example",
    prompt_version=7,
    embedding_model="text-embedding-example",
    chunk_size=700,
    chunk_overlap=100,
    retriever_type="hybrid",
    top_k=8,
    reranker_model="cross-encoder-example",
    temperature=0.1,
    index_version="support-index-sha256-a84f...",
    source_dataset_version="s3-version-id-3Lg...",
)

golden_df = pd.DataFrame(
    [
        {
            "question": "How long is the refund window?",
            "expected_answer": "Thirty days.",
            "category": "refunds",
        },
        {
            "question": "Can a customer change the delivery address?",
            "expected_answer": "Only before dispatch.",
            "category": "delivery",
        },
    ]
)

dataset = mlflow.data.from_pandas(
    golden_df,
    source=(
        "s3://company-eval-data/support/golden/v17/"
        "golden.parquet?versionId=3Lg..."
    ),
    name="support-golden-v17",
)

validation_report = {
    "schema_valid": True,
    "completeness": 0.998,
    "duplicate_fraction": 0.001,
    "freshness_timestamp": "2026-07-30T08:00:00Z",
    "validation_tool": "great-expectations",
    "validation_suite_version": "support-docs-v4",
}

with mlflow.start_run(run_name="hybrid-rag-candidate") as run:
    mlflow.log_params(asdict(config))

    mlflow.set_tags(
```

```

        {
            "git.sha": os.getenv("GIT_SHA", "local-development"),
            "image.digest": os.getenv("IMAGE_DIGEST", "not-built"),
            "environment": "development",
            "service": "support-rag-api",
            "owner": "ai-platform",
            "configuration.digest": sha256_json(asdict(config)),
        }
    )

    mlflow.log_input(dataset, context="golden-evaluation")

    mlflow.log_metrics(
        {
            "eval.correctness": 0.91,
            "eval.groundedness": 0.95,
            "eval.retrieval_relevance": 0.89,
            "eval.safety": 1.0,
            "latency.p95_ms": 1850,
            "estimated_cost_usd": 2.84,
            "error_rate": 0.003,
        }
    )

    mlflow.log_dict(
        validation_report,
        "data_quality/validation-report.json",
    )

    mlflow.log_dict(
        asdict(config),
        "lineage/rag-configuration.json",
    )

    print("Run ID:", run.info.run_id)

```

MLflow dataset records include properties such as source, name, digest, schema and profile. The dataset digest lets you detect content changes, but it does not replace an externally enforceable immutable data version. Record both the MLflow digest and the S3 object version or content hash. ([MLflow AI Platform](#))

5.4 Parent and child runs

Use nested runs for a bounded experiment such as a chunk-size and top_k sweep.

```

candidate_configs = [
    {"chunk_size": 500, "top_k": 5},
    {"chunk_size": 700, "top_k": 8},
    {"chunk_size": 900, "top_k": 10},
]

with mlflow.start_run(run_name="retrieval-sweep") as parent:
    mlflow.set_tag("experiment.type", "retrieval-sweep")

```

```

for candidate in candidate_configs:
    with mlflow.start_run(
        run_name=(
            f"chunk-{candidate['chunk_size']}"
            f"-topk-{candidate['top_k']}"
        ),
        nested=True,
    ):
        mlflow.log_params(candidate)

        # Replace with the real evaluation harness.
        simulated_score = (
            0.80
            + candidate["top_k"] * 0.005
            - abs(candidate["chunk_size"] - 700) / 10_000
        )

        mlflow.log_metric(
            "eval.retrieval_relevance",
            simulated_score,
        )

```

Use nested runs when the relationship is genuinely hierarchical. Do not create child runs merely because one service function called another; that is what tracing spans are for.

5.5 Search and compare runs

```

runs = mlflow.search_runs(
    experiment_names=["customer-support-rag"],
    filter_string=(
        "tags.environment = 'development' "
        "and metrics.`eval.correctness` >= 0.90 "
        "and metrics.`eval.safety` = 1.0"
    ),
    order_by=[
        "metrics.`latency.p95_ms` ASC",
        "metrics.`eval.correctness` DESC",
    ],
    max_results=20,
)

columns = [
    "run_id",
    "params.llm_model",
    "params.chunk_size",
    "params.top_k",
    "metrics.eval.correctness",
    "metrics.eval.retrieval_relevance",
    "metrics.latency.p95_ms",
]

```



```
print(runs[[c for c in columns if c in runs.columns]])
```

A good release gate searches for the exact candidate run ID rather than “the best recent run.” Search is for discovery and comparison; release decisions must reference immutable IDs.

6. Prompt Registry

6.1 Register a prompt

Prompt Registry versions are immutable. Aliases provide mutable lifecycle pointers, and prompt lineage integrates with traces and evaluations. ([MLflow AI Platform](#))

```
import mlflow

registered_prompt = mlflow.genai.register_prompt(
    name="support-rag-answer",
    template="""
You are a customer-support assistant.

Use only the supplied context.
If the context is insufficient, say that you do not know.
Do not invent policies.

Context:
{{context}}

Question:
{{question}}

Return a concise answer with source identifiers.
""").strip(),
    commit_message="Add explicit abstention and citation instructions",
    tags={
        "owner": "support-ai",
        "risk_class": "customer-facing",
        "evaluation_dataset": "support-golden-v17",
    },
    model_config={
        "temperature": 0.1,
        "max_tokens": 500,
    },
)

print(registered_prompt.version)
```

6.2 Assign aliases

```
mlflow.genai.set_prompt_alias(
    name="support-rag-answer",
    alias="candidate",
```

```
    version=registered_prompt.version,  
)
```

Use aliases such as:

candidate	Candidate currently under evaluation
champion	Best approved version
production	Version corresponding to the deployed production bundle
rollback	Last known-good version

Do not use `latest` as a production contract. MLflow reserves `@latest`, but it identifies the newest registration, not the most trustworthy prompt.

6.3 Load and render the prompt

```
prompt = mlflow.genai.load_prompt(  
    "prompts:/support-rag-answer@candidate"  
)  
  
rendered = prompt.format(  
    context="[doc-102] Refunds are accepted within 30 days.",  
    question="How long is the refund window?",  
)  
  
print(rendered)
```

For deterministic evaluation or deployment, load an exact version:

```
prompt = mlflow.genai.load_prompt(  
    "prompts:/support-rag-answer/7"  
)
```

Alias resolution is cached for a short period by default, while immutable version references can be cached indefinitely. The alias cache defaults to approximately 60 seconds and can be controlled through MLflow cache settings. Account for this when testing an emergency alias rollback. ([MLflow AI Platform](#))

7. GenAI tracing

7.1 Span model

A trace represents one end-to-end request. A span represents one meaningful operation.

support_rag_request	CHAIN
├─ input_guardrail	GUARDRAIL
├─ retrieve_documents	RETRIEVER
├─ rerank_documents	RERANKER
├─ build_prompt	CHAIN
├─ call_llm	LLM
├─ output_guardrail	GUARDRAIL
└─ format_response	CHAIN

MLflow traces contain trace-level metadata and span data. MLflow tracing is OpenTelemetry-compatible and supports GenAI-oriented semantics for prompts, retrievals, tools and model interactions. ([MLflow AI Platform](#))

7.2 Manual tracing

```
from __future__ import annotations

from typing import Any

import mlflow
from mlflow.entities import SpanType

@mlflow.trace(
    name="retrieve_documents",
    span_type=SpanType.RETRIEVER,
)
def retrieve_documents(question: str, top_k: int) -> list[dict[str, Any]]:
    # Replace with the vector database call.
    documents = [
        {
            "id": "doc-102",
            "text": "Refunds are accepted within 30 days.",
            "score": 0.94,
            "index_version": "support-index-sha256-a84f...",
        },
        {
            "id": "doc-205",
            "text": "Refund requests require proof of purchase.",
            "score": 0.83,
            "index_version": "support-index-sha256-a84f...",
        },
    ]
    return documents[:top_k]

@mlflow.trace(
    name="rerank_documents",
    span_type="RERANKER",
)
def rerank_documents(
    question: str,
    documents: list[dict[str, Any]],
) -> list[dict[str, Any]]:
    return sorted(
        documents,
        key=lambda document: document["score"],
        reverse=True,
    )

@mlflow.trace(
```

```

        name="call_llm",
        span_type=SpanType.LLM,
    )
def call_llm(
    prompt_text: str,
    model_name: str,
) -> str:
    span = mlflow.get_current_active_span()

    if span is not None:
        span.set_attributes(
            {
                "llm.provider": "openai",
                "llm.model": model_name,
                "llm.temperature": 0.1,
                "prompt.registry_name": "support-rag-answer",
                "prompt.version": 7,
            }
        )

    # Replace with the real model-provider or serving call.
    return "The refund window is 30 days. Source: doc-102"

@mlflow.trace(
    name="support_rag_request",
    span_type=SpanType.CHAIN,
)
def answer_question(question: str) -> str:
    mlflow.update_current_trace(
        tags={
            "service": "support-rag-api",
            "environment": "development",
            "git.sha": "abc1234",
            "image.digest": "sha256:91be...",
            "index.version": "support-index-sha256-a84f...",
            "prompt.version": "7",
        }
    )

    documents = retrieve_documents(question, top_k=8)
    reranked = rerank_documents(question, documents)

    context = "\n".join(
        f"[{document['id']}] {document['text']}"
        for document in reranked
    )

    prompt = mlflow.genai.load_prompt(
        "prompts:/support-rag-answer/7"
    )

    rendered = prompt.format(
        context=context,

```

```

        question=question,
    )

    return call_llm(
        rendered,
        model_name="gpt-example",
    )

```

The `@mlflow.trace` decorator captures function inputs, outputs, timing, exceptions and parent-child relationships. `mlflow.start_span()` is preferable when the operation does not map cleanly to one function. ([MLflow AI Platform](#))

7.3 Explicit span context

```

import mlflow
from mlflow.entities import SpanType

def run_guardrail(text: str) -> dict[str, object]:
    with mlflow.start_span(
        name="output_guardrail",
        span_type=SpanType.GUARDRAIL,
    ) as span:
        span.set_inputs({"text": text})

        result = {
            "allowed": True,
            "categories": [],
        }

        span.set_outputs(result)
        span.set_attributes(
            {
                "guardrail.policy_version": "support-policy-v9",
                "guardrail.provider": "internal",
            }
        )

    return result

```

Use attributes for low-volume operational context. Do not add raw customer identifiers or unbounded free-form text as attributes.

7.4 Autologging

For supported integrations:

```

import mlflow

mlflow.openai.autolog()

```

MLflow provides autologging for numerous model providers and agent frameworks. Use it to capture provider-level details, but retain manual spans around domain operations such as retrieval, authorization, reranking and policy enforcement. ([MLflow AI Platform](#))

Avoid accidentally instrumenting the same provider call twice. A typical pattern is:

```
Manual CHAIN span
  Manual RETRIEVER span
  Manual RERANKER span
  Automatically captured LLM span
```

7.5 Context propagation across services

Caller:

```
import requests

from mlflow.tracing import (
    get_tracing_context_headers_for_http_request,
)

headers = get_tracing_context_headers_for_http_request()

response = requests.post(
    "http://retrieval-service/retrieve",
    json={"question": "How long is the refund window?"},
    headers=headers,
    timeout=10,
)
```

Callee:

```
from fastapi import FastAPI, Request

from mlflow.tracing import (
    set_tracing_context_from_http_request_headers,
)

app = FastAPI()

@app.post("/retrieve")
def retrieve(request: Request, payload: dict):
    with set_tracing_context_from_http_request_headers(
        dict(request.headers)
    ):
        return retrieve_documents(
            payload["question"],
            top_k=8,
        )
```

MLflow uses the W3C Trace Context pattern for distributed HTTP propagation. Both services must send traces to a compatible tracking destination and preserve `traceparent`/`tracestate` headers. (MLflow AI Platform)

For Kafka, SQS, Celery or another asynchronous worker:

- 1. Inject W3C trace context into message headers.
- 2. Extract that context in the worker.
- 3. Establish it before creating the worker span.
- 4. Never assume Python thread-local context survives a process or queue boundary.
- 5. Attach the job ID, message ID and orchestrator run ID as bounded metadata.

7.6 Sampling and telemetry cost

For production:

```
export MLFLOW_TRACE_SAMPLING_RATIO=0.05
```

This retains approximately 5% of eligible traces.

A stronger strategy is:

Traffic class	Sampling
Successful normal traffic	1–5%
Errors	100%
Guardrail failures	100%
New model/prompt canary	50–100%
High-risk workflow	Policy-defined
Health checks	0%

MLflow also supports asynchronous trace logging and configuration for worker count, queueing and retries. Production applications that only require tracing can use the smaller `mlflow-tracing` package instead of the complete MLflow package, but the two packages should not be installed together in the same environment. (MLflow AI Platform)

7.7 PII redaction

Redact data **before it leaves the application process.**

```
import re
from typing import Any

import mlflow

EMAIL_PATTERN = re.compile(
    r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"
)
```

```

def redact(value: Any) -> Any:
    if isinstance(value, str):
        return EMAIL_PATTERN.sub("[REDACTED_EMAIL]", value)

    if isinstance(value, list):
        return [redact(item) for item in value]

    if isinstance(value, dict):
        return {
            key: (
                "[REDACTED]"
                if key.lower() in {"token", "password", "authorization"}
                else redact(item)
            )
            for key, item in value.items()
        }

    return value

def redact_span(span) -> None:
    if span.inputs is not None:
        span.set_inputs(redact(span.inputs))

    if span.outputs is not None:
        span.set_outputs(redact(span.outputs))

mlflow.tracing.configure(
    span_processors=[redact_span]
)

```

Client-side span processors can transform or remove sensitive fields before telemetry is sent to the backend. Validate processor behavior with representative nested payloads and failure paths. ([MLflow AI Platform](#))

For highly sensitive applications, log:

- Content classifications.
- Token counts.
- Content hashes.
- Policy outcomes.
- Document IDs.
- Truncated or encrypted payload references.

Do not log raw prompts merely because tracing makes it convenient.

8. GenAI evaluation

8.1 Current GenAI API versus classic API

Purpose	API
---------	-----

GenAI/agent evaluation	<code>mlflow.genai.evaluate()</code>
Built-in/custom GenAI scorers	<code>mlflow.genai.scorers</code>
Traditional model evaluator	<code>mlflow.evaluate()</code>
Traditional metric objects	<code>mlflow.metrics</code>
Production trace feedback	Assessments and <code>mlflow.log_feedback()</code>

The current GenAI evaluation system works with structured inputs, outputs, expectations and traces. The classic evaluation API remains useful for traditional classification, regression and compatible model-evaluator workflows, but it should not be the default mental model for new agent evaluation. ([MLflow AI Platform](#))

8.2 Evaluate the RAG application

```
from mlflow.genai.scorers import (
    Correctness,
    Guidelines,
    RelevanceToQuery,
    RetrievalRelevance,
)

evaluation_data = [
    {
        "inputs": {
            "question": "How long is the refund window?",
        },
        "expectations": {
            "expected_response": "The refund window is 30 days.",
        },
    },
    {
        "inputs": {
            "question": "Can I change the delivery address after
dispatch?",
        },
        "expectations": {
            "expected_response": (
                "No. The delivery address can only be changed "
                "before dispatch."
            ),
        },
    },
]

def predict_fn(question: str) -> str:
    return answer_question(question)

evaluation_result = mlflow.genai.evaluate(
    data=evaluation_data,
```

```

predict_fn=predict_fn,
scorers=[
    Correctness(),
    RelevanceToQuery(),
    RetrievalRelevance(),
    Guidelines(
        name="abstention_policy",
        guidelines=(
            "The answer must state that it does not know "
            "when the provided context is insufficient."
        ),
    ),
    Guidelines(
        name="citation_policy",
        guidelines=(
            "Every factual policy claim must cite at least "
            "one retrieved document identifier."
        ),
    ),
    Guidelines(
        name="safety_policy",
        guidelines=(
            "The response must not disclose credentials, "
            "private customer data, or internal-only policies."
        ),
    ),
],
)

```

RetrievalRelevance evaluates outputs from spans classified as retriever spans, which is another reason to use correct span types. ([MLflow AI Platform](#))

8.3 Custom deterministic scorers

Use deterministic code for constraints that do not require an LLM judge.

```

from mlflow.genai.scorers import scorer

@scorer
def has_source_reference(outputs) -> bool:
    if not isinstance(outputs, str):
        return False

    return "source:" in outputs.lower()

@scorer
def answer_length_within_limit(outputs) -> bool:
    if not isinstance(outputs, str):
        return False

    return len(outputs.split()) <= 120

```

Then:

```
result = mlflow.genai.evaluate(
    data=evaluation_data,
    predict_fn=predict_fn,
    scorers=[
        Correctness(),
        RetrievalRelevance(),
        has_source_reference,
        answer_length_within_limit,
    ],
)
```

Custom scorer functions may consume inputs, outputs, expectations or trace information and return scalar values or richer feedback objects. Code scorers are especially useful for schema, formatting, policy and budget constraints. ([MLflow AI Platform](#))

For latency, token and cost gates, it is often cleaner to let the evaluation harness aggregate measurements into run metrics:

```
mlflow.log_metrics(
    {
        "latency.p50_ms": 720,
        "latency.p95_ms": 1840,
        "tokens.input_total": 128_400,
        "tokens.output_total": 21_800,
        "estimated_cost_usd": 3.14,
    }
)
```

8.4 Persistent evaluation datasets

MLflow Evaluation Datasets provide managed test cases containing inputs, expectations and optional outputs. They can be created manually or populated from production traces. SQL-backed storage is required for these first-class GenAI datasets; FileStore is not sufficient. ([MLflow AI Platform](#))

A practical structure:

```
{
  "inputs": {
    "question": "Can a user receive a refund after 45 days?",
    "customer_tier": "standard"
  },
  "expectations": {
    "expected_response": "No, except for an approved exception.",
    "must_cite": ["refund-policy-v12"],
    "risk_category": "financial-policy"
  },
  "metadata": {
    "case_origin": "production_failure",
    "trace_id": "tr-...",
    "approved_by": "domain-reviewer-17",
    "added_at": "2026-07-30"
  }
}
```

```
}  
}
```

Use separate datasets for:

- Fast pull-request regression.
- Full nightly regression.
- Safety red-team cases.
- Domain-review cases.
- Production failures.
- Judge-calibration cases.

8.5 Human feedback

```
from mlflow.entities import AssessmentSource  
  
mlflow.log_feedback(  
    trace_id="tr-example123",  
    name="answer_correctness",  
    value=0.0,  
    rationale=(  
        "The answer used the old 60-day policy. "  
        "The current policy is 30 days."  
    ),  
    source=AssessmentSource(  
        source_type="HUMAN",  
        source_id="support-reviewer-42",  
    ),  
    metadata={  
        "policy_version": "refund-policy-v12",  
        "review_queue": "financial-policy",  
    },  
)
```

Human feedback is attached to a trace as an assessment and preserves reviewer, timestamp, rationale and revision information. Use the same assessment names for human labels and judge outputs when measuring judge-human agreement. ([MLflow AI Platform](#))

Judge-calibration evidence should include:

```
judge model and version  
judge prompt version  
sampling strategy  
human reviewer population  
agreement / disagreement measurements  
false-pass and false-fail analysis  
risk-category breakdown  
calibration date  
known limitations
```

Store the report as a run artifact and log summary metrics such as:

```
judge.human_agreement
judge.false_pass_rate
judge.false_fail_rate
judge.high_risk_false_pass_rate
```

8.6 Production traces to regression tests

A continuous-improvement loop:

```
Sample production traces
    ↓
Filter errors, negative feedback and policy failures
    ↓
Domain reviewer confirms expected behavior
    ↓
Add approved record to Evaluation Dataset
    ↓
Run against champion and candidate
    ↓
Diagnose prompt/retrieval/model/index failure
    ↓
Create candidate version
    ↓
CI gate and deployment
```

Do not automatically convert every thumbs-down event into ground truth. A negative rating can represent:

- Wrong answer.
- Bad source data.
- User disagreement with a correct policy.
- UX frustration.
- Malicious feedback.
- Retrieval failure.
- Latency rather than correctness.

Human validation remains the boundary between telemetry and trusted regression data.

Server-side or scheduled production scoring is deployment-dependent. Offline `mlflow.genai.evaluate()` is the portable baseline. Some scheduled scorer and monitoring workflows are associated with managed Databricks functionality, so verify availability in your chosen open-source or managed MLflow distribution before designing around them. ([MLflow AI Platform](#))

9. Model Registry and candidate promotion

9.1 Package a model or application

MLflow Model packaging is most useful when the artifact can be represented by an MLflow flavor or a Python function contract.

```

from __future__ import annotations

import pandas as pd
import mlflow
from mlflow.models import infer_signature

class SupportRAGModel(mlflow.pyfunc.PythonModel):
    def predict(
        self,
        context,
        model_input: pd.DataFrame,
        params=None,
    ):
        return [
            answer_question(question)
            for question in model_input["question"].tolist()
        ]

input_example = pd.DataFrame(
    {
        "question": [
            "How long is the refund window?"
        ]
    }
)

output_example = pd.Series(
    ["The refund window is 30 days. Source: doc-102"]
)

signature = infer_signature(
    input_example,
    output_example,
)

with mlflow.start_run(run_name="package-support-rag") as run:
    model_info = mlflow.pyfunc.log_model(
        name="support_rag_application",
        python_model=SupportRAGModel(),
        input_example=input_example,
        signature=signature,
        pip_requirements=[
            "mlflow==3.14.0",
            "pandas",
        ],
    )

```

Model signatures establish the input, output and optional inference-parameter contract. ([MLflow AI Platform](#))

Do not embed production secrets in the model artifact. Load credentials through the runtime identity and secret-management system.

9.2 Register the candidate

```
from mlflow import MlflowClient

MODEL_NAME = "support-rag-application"

model_version = mlflow.register_model(
    model_uri=model_info.model_uri,
    name=MODEL_NAME,
)

client = MlflowClient()

client.set_model_version_tag(
    name=MODEL_NAME,
    version=model_version.version,
    key="validation_status",
    value="pending",
)

client.set_model_version_tag(
    name=MODEL_NAME,
    version=model_version.version,
    key="source_run_id",
    value=run.info.run_id,
)

client.set_registered_model_alias(
    name=MODEL_NAME,
    alias="candidate",
    version=model_version.version,
)
```

Model Registry supports immutable versions, tags, descriptions and aliases. Consumers can resolve aliases such as `models:/support-rag-application@champion`, while a deterministic deployment can use `models:/support-rag-application/17`. ([MLflow AI Platform](#))

9.3 Safe promotion

Avoid this unsafe sequence:

1. Move model @production.
2. Move prompt @production.
3. Update index pointer.
4. Hope every service resolves the same combination.

Those updates are not one atomic transaction.

Use a **compatibility bundle**:

```
bundle_version: support-rag-2026-07-30.3

application:
  git_sha: abc1234
  image: registry.example.com/support-rag@sha256:91be...

model:
  registry_name: support-rag-application
  version: 17
  artifact_digest: sha256:44ab...

prompt:
  registry_name: support-rag-answer
  version: 7

embedding:
  provider: internal
  model: bge-large-en-v1.5
  revision: sha256:13fa...

data:
  source_manifest_digest: sha256:b991...
  evaluation_dataset: support-golden-v17

vector_index:
  name: customer-support
  version: sha256:a84f...
  embedding_dimension: 1024
  distance_metric: cosine

evaluation:
  mlflow_run_id: 48fd...
  correctness: 0.91
  groundedness: 0.95
  retrieval_relevance: 0.89
  safety: 1.0
  p95_latency_ms: 1850
  estimated_cost_usd: 2.84

approval:
  ticket: AI-2481
  approved_by: platform-review-board
  approved_at: 2026-07-30T10:30:00Z
```

Production promotion:

1. Resolve candidate aliases to exact versions.
2. Validate the compatibility bundle.
3. Persist approval and evaluation evidence in MLflow.
4. Commit exact versions and digests to Git.
5. Argo CD reconciles that Git commit.
6. Observe the existing rollout.

7. After successful rollout, update champion/production aliases to reflect what is actually deployed.

The runtime should consume the exact bundle values supplied through its deployment configuration. It should not re-resolve mutable aliases on every request.

9.4 Rollback

1. Revert Git to the previous compatibility bundle.
2. Argo CD restores the previous immutable deployment.
3. Confirm the old model, prompt, embedding and index combination.
4. Move aliases back to reflect the deployed state.
5. Attach the incident and rollback evidence to the affected run/version.

Alias rollback alone is insufficient when the application image, tokenizer, prompt or vector index also changed.

10. Release-gate implementation

A simplified gate:

```
from __future__ import annotations

import json
import sys

import mlflow
from mlflow import MlflowClient

MINIMUMS = {
    "eval.correctness": 0.90,
    "eval.groundedness": 0.93,
    "eval.retrieval_relevance": 0.85,
    "eval.safety": 1.0,
}

MAXIMUMS = {
    "latency.p95_ms": 2_000,
    "error_rate": 0.01,
    "estimated_cost_usd": 4.00,
}

def validate_metrics(metrics: dict[str, float]) -> list[str]:
    failures: list[str] = []

    for name, minimum in MINIMUMS.items():
        actual = metrics.get(name)

        if actual is None:
            failures.append(f"Missing required metric: {name}")
```

```

        elif actual < minimum:
            failures.append(
                f"{name}={actual:.4f} is below {minimum:.4f}"
            )

    for name, maximum in MAXIMUMS.items():
        actual = metrics.get(name)

        if actual is None:
            failures.append(f"Missing required metric: {name}")
        elif actual > maximum:
            failures.append(
                f"{name}={actual:.4f} exceeds {maximum:.4f}"
            )

    return failures

def main(run_id: str) -> None:
    client = MlflowClient()
    run = client.get_run(run_id)

    failures = validate_metrics(run.data.metrics)

    required_tags = [
        "git.sha",
        "image.digest",
        "index.version",
        "source_dataset.version",
    ]

    for tag in required_tags:
        if not run.data.tags.get(tag):
            failures.append(f"Missing lineage tag: {tag}")

    if failures:
        print(json.dumps({"status": "failed", "failures": failures},
            indent=2))
        sys.exit(1)

    print(
        json.dumps(
            {
                "status": "passed",
                "run_id": run_id,
                "git_sha": run.data.tags["git.sha"],
                "image_digest": run.data.tags["image.digest"],
                "index_version": run.data.tags["index.version"],
            },
            indent=2,
        )
    )

```

```
if __name__ == "__main__":  
    main(sys.argv[1])
```

This gate should also validate:

```
Candidate versus champion deltas  
No high-risk regression cases failed  
Judge version and calibration are approved  
Prompt version exists and is immutable  
Model artifact digest matches  
Embedding dimension matches the index  
Tokenizer is compatible with the serving model  
Input/output schema matches the application  
Required model and dataset licenses are approved  
All referenced S3 versions are readable
```

After passing, CI writes the compatibility bundle and opens or updates the environment-repository pull request. Argo CD does not query MLflow; it reconciles the immutable references committed to Git.

11. Data quality, versioning and lineage

11.1 Record validation results

MLflow should record the **results** of the existing validation process:

```
quality_metrics = {  
    "data.schema_valid": 1.0,  
    "data.completeness": 0.998,  
    "data.validity": 0.997,  
    "data.duplicate_fraction": 0.001,  
    "data.freshness_hours": 3.5,  
    "data.distribution_shift": 0.042,  
}  
  
with mlflow.start_run(run_name="validate-support-documents"):  
    mlflow.log_metrics(quality_metrics)  
  
    mlflow.log_dict(  
        {  
            "validation_suite": "support-docs-v4",  
            "failed_checks": [],  
            "source_uri": "s3://support-docs/raw/2026-07-30/",  
            "source_object_version": "3Lg...",  
            "schema_hash": "sha256:118a...",  
        },  
        "data_quality/full-report.json",  
    )
```

Do not log millions of individual failing rows as metrics. Store the detailed report as an artifact and log bounded aggregate measurements.

11.2 Lineage manifest

```
lineage_manifest = {
    "source_documents": {
        "manifest_uri": (
            "s3://support-data/manifests/"
            "documents-sha256-b991.json"
        ),
        "manifest_digest": "sha256:b991...",
        "s3_versioning_enabled": True,
        "document_count": 184_112,
    },
    "chunk_dataset": {
        "uri": (
            "s3://support-data/chunks/"
            "chunking-sha256-e810/"
        ),
        "digest": "sha256:e810...",
        "chunk_size": 700,
        "chunk_overlap": 100,
    },
    "embeddings": {
        "uri": (
            "s3://support-data/embeddings/"
            "embedding-sha256-f131/"
        ),
        "digest": "sha256:f131...",
        "model": "bge-large-en-v1.5",
        "model_revision": "sha256:13fa...",
        "dimension": 1024,
    },
    "vector_index": {
        "name": "customer-support",
        "version": "sha256:a84f...",
        "distance_metric": "cosine",
        "document_manifest_digest": "sha256:b991...",
        "embedding_digest": "sha256:f131...",
    },
    "code": {
        "git_sha": "abc1234",
        "image_digest": "sha256:91be...",
    },
}
```

Persist it:

```
with mlflow.start_run(run_name="build-support-index") as run:
    mlflow.log_dict(
        lineage_manifest,
        "lineage/end-to-end-manifest.json",
    )

    mlflow.set_tags(
```

```

    {
      "documents.digest": "sha256:b991...",
      "chunks.digest": "sha256:e810...",
      "embeddings.digest": "sha256:f131...",
      "index.version": "sha256:a84f...",
      "git.sha": "abc1234",
      "image.digest": "sha256:91be...",
    }
  )

```

Minimum viable lineage is:

```

immutable source URI or object version
content hash
validation report
code commit
container digest
chunking configuration
embedding model and revision
embedding output digest
index version
MLflow run ID
target environment

```

11.3 S3 versioning versus DVC versus lakeFS

Approach	Best fit
Immutable S3 paths + object version IDs + hashes	Small-to-medium teams, append-only pipelines, straightforward lineage
DVC	Git-centric projects where developers version datasets/models alongside code and use remote object storage
lakeFS	Large object-store datasets requiring branches, commits, merges, isolated changes and atomic promotion semantics

DVC stores lightweight metadata in Git while large data and model objects live in remote storage. lakeFS supplies Git-like repositories, immutable commits, zero-copy branches and merges over object-store data. ([GitHub](#))

Use simple S3 versioning when:

- Data is append-only.
- Every transformation writes to a new immutable prefix.
- A manifest can enumerate all inputs.
- You do not need branching or multi-object atomic merges.
- Operational simplicity matters more than a richer data-version-control interface.

Use DVC when:

- Developers need to reproduce data locally through Git commits.
- Datasets and model artifacts closely follow code branches.
- Data sizes and team workflows fit Git-oriented metadata management.

Use lakeFS when:

- Many related objects must form one data snapshot.
- Teams need isolated branches.
- A data change requires review before promotion.
- Atomic merges and rollback across a data lake are valuable.

MLflow does not replace these systems. It records which DVC revision, lakeFS commit or S3 object versions an experiment used.

12. Workflow orchestration

12.1 Choosing an orchestrator

Tool	Strongest fit
Airflow	Established scheduled batch/data platforms, explicit DAGs, mature backfills and broad operator ecosystem
Prefect	Python-first dynamic workflows, fast adoption, schedules and event-driven deployments
Kubeflow Pipelines	Kubernetes-native, containerized ML steps, GPU/custom-resource workloads and artifact-oriented pipelines

Airflow DAGs control ordering, retries, timeouts and backfills. Prefect deployments support schedules, events and remotely executed flows. Kubeflow Pipelines translates components into Kubernetes resources and supports caching, parallelism, retries and exit handling. ([Apache Airflow](#))

For this RAG example, choose **Prefect** because:

- The pipeline is primarily Python-based.
- It has scheduled and event-triggered ingestion.
- It needs retries and operational visibility.
- It does not yet require a complete Kubernetes-native ML platform.

Do not add Kubeflow solely because the application is called “AI.”

12.2 Small Prefect workflow

```
from __future__ import annotations

import hashlib
from typing import Any

import mlflow
from prefect import flow, task


def content_key(prefix: str, payload: str) -> str:
    digest = hashlib.sha256(payload.encode("utf-8")).hexdigest()
    return f"{prefix}/sha256-{digest}"
```

```

@task(
    retries=3,
    retry_delay_seconds=60,
    timeout_seconds=900,
)
def ingest(source_version: str) -> str:
    output_uri = content_key(
        "s3://support-data/ingested",
        source_version,
    )

    # Return the existing immutable output if it already exists.
    # Otherwise ingest and atomically publish it.
    return output_uri

@task(
    retries=2,
    retry_delay_seconds=30,
    timeout_seconds=600,
)
def validate(ingested_uri: str) -> dict[str, Any]:
    return {
        "valid": True,
        "source_uri": ingested_uri,
        "completeness": 0.998,
    }

@task(
    retries=2,
    timeout_seconds=1_800,
)
def chunk_and_embed(
    ingested_uri: str,
    chunk_size: int,
    overlap: int,
) -> str:
    return content_key(
        "s3://support-data/embeddings",
        f"{ingested_uri}:{chunk_size}:{overlap}:embedding-v5",
    )

@task(
    retries=2,
    timeout_seconds=1_800,
)
def publish_index(embedding_uri: str) -> str:
    return content_key(
        "vector://customer-support",
        embedding_uri,
    )

```

```

@task(timeout_seconds=1_800)
def evaluate_index(index_version: str) -> str:
    with mlflow.start_run(
        run_name="scheduled-index-evaluation",
        tags={
            "workflow": "support-index-refresh",
            "index.version": index_version,
        },
    ) as run:
        mlflow.log_metric(
            "eval.retrieval_relevance",
            0.89,
        )
        return run.info.run_id

@flow(name="support-index-refresh")
def rebuild_support_index(
    source_version: str,
    chunk_size: int = 700,
    overlap: int = 100,
) -> dict[str, str]:
    ingested_uri = ingest(source_version)
    report = validate(ingested_uri)

    if not report["valid"]:
        raise ValueError("Data validation failed")

    embedding_uri = chunk_and_embed(
        ingested_uri,
        chunk_size,
        overlap,
    )

    index_version = publish_index(embedding_uri)
    evaluation_run_id = evaluate_index(index_version)

    return {
        "index_version": index_version,
        "evaluation_run_id": evaluation_run_id,
    }

```

Deployment triggers can run the flow when a new source-manifest event arrives, and schedules can provide periodic safety-net processing. ([Prefect](#))

12.3 Production workflow requirements

Idempotency

Derive output paths from:

```

source content hash
transformation-code version

```


configuration digest embedding-model revision
--

Retries must return the same result or recognize an existing committed result.

Caching

Cache only when the complete input identity is known. A chunking cache key that omits code version or parser version is unsafe.

Backfills

Every workflow should accept an explicit source version or time interval. Do not make a historical backfill implicitly read “current latest documents.”

Timeouts

Use separate limits for:

- Document downloads.
- Parsing.
- Embedding batches.
- Index publication.
- Evaluation LLM calls.
- Registry operations.

Failure recovery

Publish an index through a temporary build identifier and change the production index pointer only after successful completion and validation.

Lineage

Pass URIs and digests between tasks. Do not pass gigabytes of embeddings through the orchestrator metadata database.

13. Docker Compose and production server configuration

13.1 Local Docker image

<pre>FROM ghcr.io/mlflow/mlflow:v3.14.0 RUN pip install --no-cache-dir \ psycpg2-binary \ boto3 \ "mlflow[auth]==3.14.0"</pre>

Pin this image after testing. In a tightly controlled environment, also pin the base image digest and Python package hashes.

13.2 Development Compose example

<pre>services: postgres: image: postgres:17</pre>

```

environment:
  POSTGRES_DB: mlflow
  POSTGRES_USER: mlflow
  POSTGRES_PASSWORD: development-only
volumes:
  - postgres-data:/var/lib/postgresql/data

minio:
  image: minio/minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: development
    MINIO_ROOT_PASSWORD: development-only
  volumes:
    - minio-data:/data

mlflow:
  build: .
  depends_on:
    - postgres
    - minio
  ports:
    - "5000:5000"
  environment:
    AWS_ACCESS_KEY_ID: development
    AWS_SECRET_ACCESS_KEY: development-only
    MLFLOW_S3_ENDPOINT_URL: http://minio:9000
  command:
    - mlflow
    - server
    - --host
    - 0.0.0.0
    - --port
    - "5000"
    - --backend-store-uri
    - postgresql://mlflow:development-only@postgres:5432/mlflow
    - --artifacts-destination
    - s3://mlflow-artifacts
    - --allowed-hosts
    - localhost:5000

volumes:
  postgres-data:
  minio-data:

```

This is development configuration. Do not copy static credentials into EKS.

13.3 Production-style command

```

mlflow server \
  --host 0.0.0.0 \
  --port 5000 \
  --workers 4 \

```

```
--backend-store-uri "${MLFLOW_DATABASE_URI}" \  
--artifacts-destination "s3://company-mlflow-prod/artifacts" \  
--allowed-hosts "mlflow.internal.example.com" \  
--app-name basic-auth
```

When binding publicly or to `0.0.0.0`, configure allowed hosts. Modern MLflow server middleware added host validation to defend against DNS-rebinding-style attacks. ([MLflow AI Platform](#))

Basic auth is available by installing `mlflow[auth]`, setting the server secret and starting the corresponding authentication application. For larger production environments, integrate an organizational identity provider or an appropriate authentication plugin rather than managing long-lived local passwords. ([MLflow AI Platform](#))

14. AWS and EKS integration

14.1 Core deployment

MLflow server pods	EKS private subnets
PostgreSQL metadata	RDS PostgreSQL Multi-AZ
Artifacts	Versioned, encrypted S3 bucket
Authentication	Internal IdP/OIDC integration
Pod AWS access	EKS IAM role for service account
Ingress	Internal ALB/NLB or approved ingress
Secrets	External Secrets/Secrets Manager
Metrics	Prometheus ServiceMonitor
Backups	RDS PITR + S3 versioning/lifecycle

Use EKS IAM roles for service accounts so the MLflow pod receives temporary AWS credentials through its Kubernetes service account rather than static access keys. ([AWS Documentation](#))

Example service account:

```
apiVersion: v1  
kind: ServiceAccount  
metadata:  
  name: mlflow  
  namespace: ml-platform  
  annotations:  
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/mlflow-prod
```

The associated role should have only the necessary S3 access.

14.2 Artifact permission warning

With proxied artifact serving, the MLflow server uses its own storage credentials. Anyone authorized to access that MLflow server may indirectly gain access to artifacts available through it.

Therefore:

- Separate sensitive tenants with separate buckets or instances.
- Restrict artifact prefixes.
- Use separate IAM roles for production and non-production.
- Avoid one powerful MLflow role across every business unit.

- Consider direct artifact access when per-user storage authorization is required.

MLflow's documentation explicitly warns that proxied artifact access can effectively grant the server's artifact privileges to users who can access the tracking service. ([MLflow AI Platform](#))

14.3 Helm and Kubernetes

Recent MLflow releases include an official production-oriented Helm chart with support for ingress, TLS, Prometheus metrics, ServiceMonitor, NetworkPolicy, RBAC and garbage-collection jobs. ([MLflow AI Platform](#))

Conceptual values:

```
replicaCount: 3

backendStore:
  databaseMigration: true
  postgres:
    enabled: false
    host: mlflow-prod.cluster-xyz.ap-south-1.rds.amazonaws.com
    database: mlflow

artifactRoot:
  s3:
    enabled: true
    bucket: company-mlflow-prod
    path: artifacts

serviceAccount:
  create: true
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/mlflow-prod

ingress:
  enabled: true
  className: alb

networkPolicy:
  enabled: true

serviceMonitor:
  enabled: true
```

Keep database credentials outside values files. Inject them through your established secret flow.

15. MLflow and production observability

What MLflow should track

```
Experiment configuration
Evaluation results
Dataset and index identity
Model and prompt lineage
```

Detailed sampled GenAI traces Human and judge feedback Release approvals Artifact and registry history

What Prometheus/Grafana/OpenTelemetry should monitor

Request rate Error rate Latency histograms CPU, memory and GPU Queue depth Pod restarts Database connections S3 or vector-store failures Saturation Alerting and SLO burn rates
--

OpenTelemetry provides vendor-neutral tracing, metrics and log context propagation. Prometheus is designed for time-series metrics and alerting, while Grafana provides dashboards and exploratory visualization. MLflow traces can coexist with an OpenTelemetry-based operational stack rather than replacing it. ([OpenTelemetry](#))

A good split:

Prometheus alert: RAG p95 latency exceeded 2.5 seconds for 15 minutes.
MLflow investigation: Slow traces correlate with prompt v8, top_k=20, reranking enabled and index version 3f91.

Do not put `user_id`, `question`, `document_id` or `trace_id` into Prometheus labels. That creates dangerous cardinality growth. Preserve detailed identifiers in sampled traces instead.

16. Serving boundaries

MLflow serving is sufficient when

- The model has an MLflow flavor or PyFunc contract.
- Python-based preprocessing and postprocessing are central.
- Traffic and latency requirements are moderate.
- A simple REST inference server is appropriate.
- The artifact contains all required reproducible dependencies.
- You are testing or serving conventional ML models.

Use a specialized serving platform when

- Serving large GPU-based language models.
- Continuous batching is required.
- Tensor parallelism or specialized scheduling is required.

- You need model-mesh capabilities.
- You need advanced autoscaling or multi-model routing.
- The provider model is external and cannot be packaged locally.

MLflow's responsibility then becomes:

Register approved artifact or external model identity
 Record evaluation and approval evidence
 Record tokenizer and runtime requirements
 Expose exact immutable version
 Hand it to the existing serving platform

MLflow 3.x also supports external-model concepts for tracking systems whose weights are not stored as an MLflow model artifact, including provider-hosted or externally served models. ([MLflow AI Platform](#))

Compatibility should cover:

Component	Required compatibility
Application image	Request/response schema
Base model	Serving runtime and hardware
Adapter	Base-model revision
Tokenizer	Model vocabulary and chat template
Prompt	Expected variables and output contract
Embedding model	Vector dimensions and preprocessing
Vector index	Embedding revision and distance metric
Guardrail	Output schema and policy version
Evaluation set	Domain and policy version

17. Production concerns

17.1 Drift

Drift type	Evidence	Likely action
Source-data drift	Schema/distribution/freshness reports	Fix ingestion or review data
Retrieval drift	Recall/relevance decline	Re-index or change retrieval
Prompt drift	Quality changes by prompt version	Roll back or revise prompt
Model drift	Answer behavior changes	Change model or adapter
Feedback drift	Rating distribution changes	Investigate users/UI/data
Judge drift	Human-judge agreement declines	Recalibrate or replace judge
Policy drift	Domain rules change	Update source, tests and prompt

A failed answer does not automatically imply retraining:

Missing documents	→ fix ingestion
Stale vectors	→ re-index
Poor retrieval ranking	→ retriever/reranker change
Prompt ambiguity	→ prompt version
Model capability issue	→ model change/fine-tuning
Runtime regression	→ application rollback

17.2 Multi-tenancy

Logical workspace and RBAC capabilities have expanded in recent MLflow releases. For strict regulatory or contractual isolation, prefer separate MLflow instances, databases, S3 prefixes/buckets, KMS keys and IAM roles rather than relying solely on UI-level organization. ([MLflow AI Platform](#))

17.3 Licensing and governance

For every candidate, attach:

- Model card
- Dataset documentation
- Source-document ownership
- Model license
- Dataset/content license
- Embedding-model license
- Known limitations
- Bias/fairness results
- Risk classification
- Privacy assessment
- Approval record
- Evaluation report

Example tags:

```
client.set_model_version_tag(
    MODEL_NAME,
    model_version.version,
    "license.model",
    "approved-license-id",
)

client.set_model_version_tag(
    MODEL_NAME,
    model_version.version,
    "governance.approval_ticket",
    "AI-2481",
)

client.set_model_version_tag(
    MODEL_NAME,
    model_version.version,
    "risk.assessment_version",
    "support-risk-v6",
)
```

Tags are evidence pointers, not a substitute for review. Store complete signed or immutable reports as artifacts.

17.4 Retention and lifecycle

Define separate policies:

Data	Example retention
Normal sampled traces	30–90 days
Error and safety traces	180–365 days
Release evaluation evidence	Product lifetime plus policy period
Model and prompt versions	Product/audit requirements
Temporary sweep artifacts	30 days
Unapproved large artifacts	30–90 days
Approval and rollback evidence	Long-term

Newer workspace functionality supports trace archival configuration. Regardless of product capability, define matching metadata retention, S3 lifecycle, legal hold and deletion workflows. ([MLflow AI Platform](#))

17.5 High availability and backup

Production checklist:

```
Multiple stateless MLflow server replicas
External PostgreSQL
Database backups and point-in-time recovery
S3 versioning and lifecycle rules
Encrypted connections
Encrypted storage
Readiness/liveness probes
Connection-pool limits
Schema migration procedure
Backup before MLflow upgrades
Artifact integrity checks
Garbage-collection policy
Disaster-recovery test
```

Database and artifacts must be backed up together from a lineage perspective. Restoring only PostgreSQL may leave registry records pointing to missing artifacts. Restoring only S3 may leave artifacts with no searchable metadata.

18. Common mistakes

1. Using mutable aliases directly at runtime

Problem: Replicas can resolve different versions during an alias update.

Fix: Resolve the alias during release creation, commit an immutable version to Git and restart/reconcile the deployment.

2. Treating a model version as the complete RAG release

Problem: The prompt, embedding model and index may be incompatible.

Fix: Promote a compatibility bundle.

3. Storing artifacts on pod-local disk

Problem: Artifacts disappear after rescheduling and replicas see inconsistent files.

Fix: Use S3 or another durable shared artifact store.

4. Using SQLite for a busy multi-user server

Problem: Locking and concurrency limitations create unstable tracking.

Fix: Use PostgreSQL for production.

5. Missing artifacts

Check:

Does the run's artifact URI exist?
Does the server role have s3:GetObject?
Is the bucket/prefix correct?
Did the client use direct or proxied artifact access?
Does the object version still exist?
Was an S3 lifecycle rule too aggressive?

6. Unreproducible experiment

Typical missing identities:

Git SHA
Docker digest
Prompt version
Source-data version
Embedding revision
Index version
Dependency lock
Random seed
Provider model version

Fail the release gate when required lineage fields are absent.

7. Offline pass, production failure

Likely causes:

- Evaluation set is unrepresentative.

- Prompt or index differs from evaluation.
- Provider behavior changed.
- Production context is longer.
- Tool permissions differ.
- Timeout/retry behavior changes outputs.
- User distribution shifted.
- Judge tolerated a serious domain error.

Compare exact production traces against the candidate evaluation bundle.

8. Broken trace propagation

Symptoms:

- Every microservice creates a separate root trace.
- Retrieval spans are missing from the parent request.
- Worker jobs are disconnected.

Check W3C propagation headers, ingress/header filtering, message-header serialization and context establishment before creating spans.

9. Runaway trace volume

Causes:

- Sampling ratio accidentally set to 1.
- Health checks traced.
- Large documents captured in every span.
- Autologging and manual instrumentation duplicate calls.
- Unbounded attributes.

Use endpoint-specific sampling, payload truncation, redaction and retention policies.

10. Duplicate indexes after retries

Cause: Non-idempotent task writes to a random destination on every retry.

Fix: Derive the output ID from input content and transformation configuration, write to a temporary location and atomically publish once.

11. Alias promoted incorrectly

Fix:

1. Record previous alias target.
 2. Validate requested version.
 3. Verify approval tags.
 4. Change alias.
 5. Read it back.
 6. Log the operation.
 7. Keep Git deployment references immutable.
-

12. Unsafe prompt logging

Never log:

- Authorization headers.
- API keys.
- Full customer records.
- Payment information.
- Unredacted health or identity data.
- Complete confidential documents.

Log bounded, redacted samples or encrypted references governed by access policy.

19. Complete release lifecycle

1. Orchestrator ingests an immutable source-data version.
 2. Validation results and manifest are logged to MLflow.
 3. Chunking and embeddings create immutable output versions.
 4. Vector index is built under an immutable version.
 5. Candidate prompt is registered.
 6. Candidate model/application is logged and registered.
 7. Golden and regression evaluations execute.
 8. RAG traces capture retrieval, reranking, LLM and guardrail behavior.
 9. Human and judge-calibration evidence is attached.
 10. Candidate-versus-champion evidence is persisted in one MLflow run.
 11. CI reads that exact run and applies release thresholds.
 12. An approver validates risk, licensing and domain evidence.
 13. CI produces an immutable compatibility bundle.
 14. Exact image/model/prompt/index references are committed to Git.
 15. Argo CD reconciles the cluster.
 16. Operational systems monitor the rollout.
 17. Production traces and feedback are sampled.
 18. Confirmed failures become evaluation-dataset cases.
 19. Champion/production aliases are updated to reflect deployed reality.
 20. Rollback reverts the Git compatibility bundle and then aliases.
-

20. Hands-on exercises

Exercise 1 – Retrieval experiment

Run four candidates:

```
chunk_size: 400 or 800  
top_k: 5 or 10
```

For each child run, log:

```
Retrieval relevance  
Correctness  
P95 latency  
Average context tokens
```

Estimated cost Index digest

Select a candidate only if it improves retrieval without violating latency or cost limits.

Exercise 2 – Prompt promotion and rollback

1. Register prompt versions 1 and 2.
 2. Point champion at version 1.
 3. Point candidate at version 2.
 4. Evaluate both on the same dataset.
 5. Promote version 2.
 6. Simulate a production failure.
 7. Roll the Git bundle and prompt alias back to version 1.
 8. Confirm the immutable prompt reference used by the application.
-

Exercise 3 – Production failure to regression case

1. Generate a traced RAG request with stale policy context.
 2. Attach human feedback explaining the error.
 3. Add the reviewed trace to an Evaluation Dataset.
 4. Rebuild the index from the current source documents.
 5. Evaluate old and new index versions.
 6. Persist the comparison run.
 7. Generate the compatibility bundle that contains the new index digest.
-

21. Interview Q&A

1. What is MLflow's core responsibility in a production RAG system?

MLflow provides lifecycle evidence and lineage: runs, datasets, artifacts, prompts, models, traces, evaluations, feedback and registry metadata. It does not replace orchestration, GitOps deployment or real-time infrastructure monitoring.

2. Why use PostgreSQL and S3 separately?

PostgreSQL stores transactional searchable metadata. S3 stores large immutable artifacts. This allows the tracking server to remain stateless and scale horizontally.

3. Why are Model Registry stages no longer the recommended approach?

Stages combine lifecycle state with a globally mutable pointer and become awkward across multiple environments. Aliases, tags and separate deployment environments provide more flexible, auditable promotion patterns.

4. How would you version a complete RAG release?

I would create an immutable compatibility bundle containing the application image digest, model version, tokenizer, prompt version, embedding revision, source-data digest, vector-index version and evaluation run ID.

5. Should the production application resolve @production on every request?

No. The release process should resolve the alias once and deploy an exact immutable version. Continuous alias resolution can cause replicas to use inconsistent combinations.

6. How do MLflow runs differ from traces?

A run represents an experiment, pipeline execution or evaluation unit and aggregates parameters, metrics and artifacts. A trace represents one request. Its spans describe operations such as retrieval, reranking, tools, LLM calls and guardrails.

7. How do you trace across microservices?

Propagate W3C Trace Context headers, extract them in the downstream service before starting spans and send the related spans to compatible MLflow tracking configuration. For queues, propagate the same context through message headers.

8. How do you control the cost and privacy of production tracing?

Use sampling by traffic class, client-side PII redaction, payload truncation, bounded attributes, asynchronous export and explicit retention policies. Preserve all errors and safety events while sampling ordinary successful traffic.

9. What is the difference between `mlflow.genai.evaluate()` and `mlflow.evaluate()`?

`mlflow.genai.evaluate()` is the current interface for LLM and agent evaluation using structured inputs, expectations, traces and GenAI scorers. `mlflow.evaluate()` is the classic evaluator framework used mainly for traditional model evaluation and legacy-compatible workflows.

10. How do you prevent LLM-as-judge drift?

Version the judge model and prompt, maintain human-labeled calibration data, measure agreement and false-pass rates by risk category and block release when calibration becomes unacceptable.

11. When would you use DVC or lakeFS?

DVC is useful for Git-centered projects that version large data and model files through remote storage. lakeFS is useful when object-store data needs branches, immutable commits, merges and atomic data promotion. Simple immutable S3 paths are enough for less complex append-only workflows.

12. Why is MLflow not a workflow scheduler?

It does not manage task dependencies, schedules, retries, backfills or worker execution. An orchestrator runs the workflow; MLflow records the workflow's inputs, outputs and evidence.

13. How does CI consume MLflow evidence?

CI receives an exact candidate run ID, retrieves required metrics and lineage tags, checks candidate-versus-champion thresholds and produces an immutable release bundle only when all technical and governance gates pass.

14. How does Argo CD integrate with MLflow?

Argo CD does not need to query MLflow. CI converts approved MLflow evidence into exact image, model, prompt and index references committed to Git. Argo CD reconciles that Git state.

15. A model passes offline evaluation but fails in production. What do you investigate?

I compare the deployed compatibility bundle with the evaluated bundle, inspect production traces, verify data and index freshness, check provider and tool behavior, examine traffic differences and validate whether the evaluation set and judge captured the failing scenario. I then add a human-confirmed failure case to the regression dataset.

Final mental model

MLflow answers:

"What exactly did we build, evaluate, approve and observe?"

The orchestrator answers:

"When and in what order should the work run?"

CI answers:

"Is this immutable candidate releasable?"

Git answers:

"What exact versions should this environment run?"

Argo CD answers:

"Does the cluster match that declared state?"

The serving platform answers:

"How is inference executed efficiently?"

The observability stack answers:

"Is the production system healthy right now?"